

SBA301 – SLOT 17

CHAPTER 13 – DATA PERSISTENCE WITH JPA

PART A: JPA, ENTITY, REPOSITORY, SERVICE + RELATIONSHIPS + PA02

Tư duy trung tâm của Slot 17

REST API chỉ mô tả cách hệ thống giao tiếp. JPA giúp ánh xạ mô hình đối tượng Java sang dữ liệu quan hệ; Repository cung cấp abstraction truy cập dữ liệu; Service đặt business logic và transaction boundary; relationship mapping quyết định cách các entity liên kết và cách foreign key/join table được tạo.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	17
Chủ đề	Chapter 13 Part A – JPA, Entity, Repository, Service, Relationships + PA02
CLO chính	CLO4 – Apply RESTful APIs with data persistence/JPA in the course sequence
Môi trường	JDK 21 • Spring Boot 3.x track • Maven • Spring Data JPA • H2 demo / SQL Server bridge
Thời lượng gợi ý	135 phút trên lớp + tự học/thực hành ngoài giờ

Student Learning Guide • Design -> Map -> Persist -> Query -> Verify -> Explain

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

Entry condition

Sinh viên phải có một Spring Boot project đã chạy được từ Slot 13–16. Nếu Lab 03 chưa ổn định, hãy sửa build/test trước khi thêm JPA để tránh trộn lỗi REST với lỗi persistence.

0.1. Kiến thức cần ôn lại

- 3-layer architecture: Controller -> Service -> Repository.
- REST annotations, request/response, status code và global exception handling.
- Paging/Sorting với Pageable, Page và Slice.
- Testing cơ bản với JUnit 5, Mockito, MockMvc; hiểu rằng @DataJpaTest chỉ dành cho JPA slice.
- Khái niệm database: table, row, primary key, foreign key, 1-N, N-N, normalization ở mức cơ bản.

0.2. Công cụ và project baseline

Thành phần	Yêu cầu	Kiểm tra nhanh
JDK	21	java -version
Maven	3.6.3+ phù hợp Spring Boot 3.x	mvn -version
IDE	IntelliJ IDEA / VS Code	Project import không lỗi
Spring Boot	3.x course track	pom.xml dùng parent/BOM của Spring Boot
Dependencies	Spring Web, Spring Data JPA, Validation, H2 (demo), Test	Maven reload xanh
Database bridge	SQL Server sẽ dùng trong project phase tiếp theo	Có thể cài sẵn SSMS/SQL Server nhưng chưa bắt buộc cho demo
Git	Repository và branch cá nhân	git status / git log

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
Repository là gì? Entity khác DTO thế nào? Quan hệ DB biểu diễn bằng gì?	Ai sở hữu quan hệ? Khi nào cần mappedBy?	Ví dụ: Vì sao @ManyToMany thường cần join table?

0.4. Pre-flight checklist

- Spring Boot project chạy và test hiện tại pass trước khi thêm dependency mới.
- Không còn lỗi compilation từ Slot 16.
- Đã commit trạng thái baseline: tag/commit gợi ý s17-before-jpa.
- Biết mở H2 console hoặc ít nhất biết xem log SQL trong console.
- Mang theo ERD/API contract của Group Project để dùng trong PA02.

I. TÓM TẮT CHỦ ĐỀ & MỤC TIÊU BÀI HỌC

1.1. Slot 16 -> Slot 17: bước chuyển từ API sang dữ liệu bền vững

Nền tảng trước Slot 17	Đã làm được	Vấn đề mới cần giải quyết
REST Controller	Nhận request, trả response	Dữ liệu thật được lưu ở đâu?
Service	Điều phối business logic	Business operation có transaction boundary thế nào?
In-memory Repository	Demo CRUD nhanh	Restart app là mất dữ liệu; không có relational constraints
Paging/Sorting	Hiểu Page/Slice	Cần database-backed queries thay vì cắt List thủ công

Nền tảng trước Slot 17	Đã làm được	Vấn đề mới cần giải quyết
Testing	Test controller/service	Cần JPA slice test khi repository thật xuất hiện

1.2. Mục tiêu học tập

- Giải thích JPA, ORM, Hibernate và Spring Data JPA khác nhau thế nào.
- Tạo entity hợp lệ với @Entity, @Id, @GeneratedValue, @Column và các kiểu dữ liệu phù hợp.
- Thiết kế repository bằng JpaRepository và hiểu vì sao không cần tự viết implementation CRUD cơ bản.
- Đặt business logic vào Service và nhận diện transaction boundary.
- Ánh xạ đúng quan hệ Many-to-One / One-to-Many / Many-to-Many ở mức nền tảng.
- Giải thích owning side, mappedBy, foreign key, join table, cascade, orphanRemoval và fetch strategy.
- Dùng H2 để kiểm chứng mapping và SQL được sinh ra; chuẩn bị chuyển sang SQL Server ở các slot tiếp theo.
- Áp dụng kiến thức vào ERD/API contract của Group Project trong PA02.

1.3. Mental model của JPA

Mental model

```

Java object graph
| @Entity / relationship annotations
v
JPA metadata
| Hibernate (JPA provider)
v
SQL statements <---- transaction boundary ----> Persistence Context
|
v
Relational Database (tables, PK, FK, join tables)

```

Không đồng nhất các khái niệm

JPA là specification/API; Hibernate là một JPA provider/ORM implementation; Spring Data JPA xây repository abstraction trên JPA để giảm boilerplate. Spring Boot auto-configures phần lớn plumbing khi dependency và datasource phù hợp.

1.4. Flow tổng quát

Request-to-database flow

```

HTTP request
-> Controller (Slot 13)
-> Service (business rule + transaction boundary)
-> Repository (Spring Data JPA abstraction)
-> EntityManager / Hibernate
-> SQL
-> Database
-> result entity / DTO
-> response

```

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG PHẢI NẮM VỮNG

2.1. JPA, ORM, Hibernate, Spring Data JPA

Khái niệm	Vai trò	Không nên hiểu sai
ORM	Kỹ thuật ánh xạ object <-> relational data	ORM không loại bỏ việc phải hiểu SQL/DB

Khái niệm	Vai trò	Không nên hiểu sai
JPA / Jakarta Persistence	Chuẩn API/annotation cho persistence	JPA không phải database và không phải framework web
Hibernate ORM	JPA provider phổ biến, sinh SQL và quản lý persistence context	Hibernate không đồng nghĩa toàn bộ JPA
Spring Data JPA	Repository abstraction cho JPA	Không phải “một database mới”; nó dùng JPA provider phía dưới
JpaRepository	Interface cung cấp CRUD + paging/sorting và JPA-specific operations	Không cần tự implement CRUD cơ bản

2.2. Entity là gì?

Entity là một đối tượng domain có identity và lifecycle được persistence provider quản lý. Trong mô hình quan hệ, entity thường được ánh xạ tới một table và mỗi instance tương ứng một row. Điểm quan trọng không phải chỉ là gắn @Entity, mà là thiết kế identity, constraints và associations phù hợp với business domain.

Annotation	Ý nghĩa	Ví dụ/ghi chú
@Entity	Đánh dấu persistent entity	Class cần constructor không tham số có thể truy cập bởi provider
@Table	Tùy biến tên table/schema	@Table(name="news")
@Id	Primary key của entity	Long id
@GeneratedValue	Chiến lược sinh khóa	IDENTITY/AUTO tùy database/strategy
@Column	Tên cột/nullable/length/unique...	@Column(nullable=false,length=150)
@Enumerated	Map enum sang DB	Ưu tiên EnumType.STRING khi phù hợp
@Transient	Không persist field	Derived/calculated property

Ví dụ 1 – Entity cơ bản

```
package com.example.funews.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "categories")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(length = 500)
    private String description;

    @Column(nullable = false)
    private boolean active = true;

    protected Category() {}

    public Category(String name, String description) {
```

```

    this.name = name;
    this.description = description;
}

// getters/setters
}

```

2.3. Entity state và Persistence Context – mức nền tảng

State	Ý nghĩa	Ví dụ
Transient	Object mới, chưa được persistence context quản lý	<code>new Category(...)</code>
Managed/Persistent	Đang được EntityManager/Hibernate theo dõi	<code>entity load trong transaction</code>
Detached	Từng managed nhưng không còn nằm trong persistence context	sau khi context đóng/clear
Removed	Được đánh dấu sẽ xóa	<code>delete/remove trong transaction</code>

Dirty checking

Trong một transaction, khi entity đang managed, thay đổi field có thể được Hibernate phát hiện và đồng bộ xuống database khi flush/commit. Vì vậy “gọi `save()` sau mọi setter” không phải lúc nào cũng là yêu cầu kỹ thuật; cần hiểu entity state và transaction.

2.4. Repository abstraction

Ví dụ 2 – Repository

```

package com.example.funews.repository;

import com.example.funews.domain.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface CategoryRepository extends JpaRepository<Category, Long> {
    Optional<Category> findByNameIgnoreCase(String name);
    boolean existsByNameIgnoreCase(String name);
}

```

- Spring Data tạo implementation của repository interface tại runtime; sinh viên không viết `CategoryRepositoryImpl` cho CRUD cơ bản.
- Tên query method có semantics: `findBy...`, `existsBy...`, `countBy...`; cần đặt tên đúng property.
- Repository là data access boundary, không nên chứa rule nghiệp vụ như “không cho xóa category đang được dùng”. Rule đó thuộc Service.

2.5. Service layer và transaction boundary

Ví dụ 3 – Service

```

package com.example.funews.service;

import com.example.funews.domain.Category;
import com.example.funews.repository.CategoryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class CategoryService {
    private final CategoryRepository categoryRepository;
}

```

```

public CategoryService(CategoryRepository categoryRepository) {
    this.categoryRepository = categoryRepository;
}

@Transactional
public Category create(String name, String description) {
    if (categoryRepository.existsByNameIgnoreCase(name)) {
        throw new IllegalArgumentException("Category name already exists");
    }
    return categoryRepository.save(new Category(name, description));
}

@Transactional(readOnly = true)
public Category getRequired(Long id) {
    return categoryRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Category not found: " + id));
}
}

```

Nguyên tắc thiết kế

Controller nên điều phối HTTP; Repository truy cập dữ liệu; Service là nơi tự nhiên để đặt business rule và transaction boundary khi một use case có thể liên quan nhiều repository/entity.

2.6. Relationships: từ ERD sang object graph

ERD / Database	JPA mapping	Ý nghĩa
N News thuộc 1 Category	News @ManyToOne Category	FK category_id thường nằm ở bảng news
1 Category có nhiều News	Category @OneToMany(mappedBy="category")	Inverse side trong bidirectional mapping
N News có N Tag	@ManyToMany + @JoinTable	Join table news_tags
1 Account có 1 Profile	@OneToOne	Unique FK hoặc shared key tùy design

Ví dụ 4 – Owning side Many-to-One

```

@Entity
@Table(name = "news")
public class News {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    protected News() {}

    public News(String title, Category category) {
        this.title = title;
        this.category = category;
    }
}

```

Ví dụ 5 – Inverse side One-to-Many + helper methods

```

@Entity
@Table(name = "categories")
public class Category {
    // id, name, ...

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<News> newsItems = new ArrayList<>();

    public void addNews(News news) {
        newsItems.add(news);
        news.setCategory(this);
    }

    public void removeNews(News news) {
        newsItems.remove(news);
        news.setCategory(null);
    }
}

```

Owning side

Trong quan hệ bidirectional One-to-Many/Many-to-One, phía @ManyToOne thường là owning side vì chứa foreign key. mappedBy trên @OneToMany trỏ tới field sở hữu quan hệ ở entity con; mappedBy không phải tên cột DB.

2.7. Many-to-Many và join table

Ví dụ 6 – Many-to-Many

```

@Entity
@Table(name = "tags")
public class Tag {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true)
    private String name;

    @ManyToMany(mappedBy = "tags")
    private Set<News> newsItems = new HashSet<>();
}

// in News
@ManyToMany
@JoinTable(
    name = "news_tags",
    joinColumns = @JoinColumn(name = "news_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();

```

Khi nào KHÔNG nên dùng @ManyToMany trực tiếp

Nếu quan hệ có thuộc tính riêng (ví dụ assignedAt, quantity, roleInProject), hãy mô hình hóa join table thành một entity trung gian thay vì @ManyToMany thuần. Đây là bước chuyển quan trọng từ “annotation demo” sang domain model thực tế.

2.8. Cascade, orphanRemoval và fetch – hiểu trước khi dùng

Khái niệm	Câu hỏi cần trả lời	Cảnh báo
cascade	Operation trên parent có nên lan sang child?	Không dùng CascadeType.ALL theo thói quen, đặc biệt với shared entities

Khái niệm	Câu hỏi cần trả lời	Cảnh báo
orphanRemoval	Child bị gỡ khỏi collection có nên bị xóa khỏi DB?	Chỉ phù hợp khi lifecycle child thực sự thuộc parent
LAZY	Chỉ lấy association khi cần	Access ngoài persistence context có thể gây lỗi/extra query
EAGER	Association phải được lấy sớm	Dễ kéo dữ liệu không cần và tạo performance problem
N+1 awareness	Một query parent có kích hoạt N query child không?	Chưa tối ưu sâu ở Part A nhưng phải nhận diện

III. ỨNG DỤNG THỰC TIỄN TRONG SẢN PHẨM

3.1. Từ database schema đến domain model

Hệ thống	Entity ví dụ	Relationships
E-commerce	Product, Category, Order, OrderItem, User	Category 1-N Product; Order 1-N OrderItem; Product 1-N OrderItem
LMS	Course, Student, Enrollment, Instructor	Student N-N Course thông qua Enrollment entity
Booking	Customer, Service, Appointment, Staff	Customer 1-N Appointment; Staff 1-N Appointment
FUNews	News, Category, Tag, Account	Category 1-N News; News N-N Tag; Account 1-N News

3.2. Case study xuyên suốt – FUNews persistence

Relationship sketch

```

Category 1 ----- N News N ----- N Tag
      |
      N
      |
      1
    Account
  
```

Database view:

- categories(id, name, ...)
- news(id, title, category_id, author_id, ...)
- tags(id, name)
- news_tags(news_id, tag_id)
- accounts(id, ...)

Mini design decision – News và Tag

Mục tiêu: Chọn mô hình phù hợp trước khi code annotation.

- Nếu tag chỉ có id/name và không có thuộc tính của quan hệ -> Many-to-Many có thể phù hợp.
- Nếu cần addedBy, addedAt, relevanceScore cho mỗi News-Tag -> tạo entity NewsTag.
- Vẽ lại ERD cho cả hai phương án và nêu trade-off.

Bằng chứng cần lưu: Ảnh ERD + 3–5 dòng giải thích quyết định.

IV. HƯỚNG DẪN CHI TIẾT THỰC HIỆN VÀ TRIỂN KHAI

A. Thiết lập project JPA demo

Để cô lập việc học mapping khỏi SQL Server configuration, demo trên lớp dùng H2 in-memory. Sau khi mapping đúng, nhóm project chuyển cùng entity/repository logic sang SQL Server ở phase database integration.

pom.xml – dependencies chính

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

```

application.properties – demo H2

```

spring.datasource.url=jdbc:h2:mem:funewsd
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

Không mang create-drop vào production

create-drop phù hợp cho demo/test vì schema bị tạo/xóa theo lifecycle ứng dụng. Khi chuyển sang SQL Server/project thật, cần chọn schema-management strategy có kiểm soát và tránh mất dữ liệu.

B. Bước 1 – Thiết kế identity và constraints trước annotation

Câu hỏi	Ví dụ Category	Quyết định
Entity có identity gì?	id Long	Generated surrogate key
Field nào bắt buộc?	name	nullable=false
Field nào unique?	name	unique=true nếu business yêu cầu
Length thực tế?	name 100	@Column(length=100)
Có status?	active	boolean default true

Worksheet 1 – Entity Design

Mục tiêu: Thiết kế entity trước khi code annotation.

- Chọn một entity của Group Project.
- Liệt kê identity, required fields, unique constraints, value ranges và relationships.
- Đánh dấu field thuộc persistence, field chỉ dùng DTO/UI và field derived.

Bảng chứng cần lưu: Bảng Entity Design 1 trang.

C. Bước 2 – Tạo entities và verify schema

Category.java

```

@Entity
@Table(name = "categories")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(length = 500)
    private String description;

    @Column(nullable = false)
    private boolean active = true;

    protected Category() {}
    // constructor, getters, setters
}

```

1. Run application và đọc log DDL/SQL thay vì chỉ nhìn “Started ...”.
2. Mở H2 console, kiểm tra table CATEGORIES và constraints được tạo.
3. Tạo lỗi có chủ đích: bỏ @Id và quan sát startup failure; sau đó khôi phục.

Break-It 1 – Entity Mapping

Mục tiêu: Học từ lỗi mapping startup.

- Xóa tạm @Id hoặc tạo hai @Column cùng tên.
- Đọc root cause trong stack trace.
- Ghi lại: lỗi ở compile time, startup time hay runtime? Vì sao?

Bảng chứng cần lưu: Ảnh lỗi + root cause + fix commit.

D. Bước 3 – Repository và derived query

CategoryRepository.java

```

public interface CategoryRepository extends JpaRepository<Category, Long> {
    Optional<Category> findByNameIgnoreCase(String name);
    List<Category> findByActiveTrueOrderByNameAsc();
    boolean existsByNameIgnoreCase(String name);
}

```

Method	Spring Data hiểu gì?	Use case
findById(id)	CRUD method có sẵn	Get required/optional entity
save(entity)	Insert/update tùy entity state/identifier	Create/update
deleteById(id)	Delete theo id	Delete operation
findByNameIgnoreCase	Derived query từ property name	Search exact case-insensitive
findByActiveTrueOrderByNameAsc	Filter + sort từ method name	Danh sách category active

Repository API surface

Không phải lúc nào interface càng “to” càng tốt. Spring Data tài liệu hiện hành cũng nhấn mạnh cân nhắc mức API surface cần expose. Trong khóa học, JpaRepository phù hợp vì sinh viên cần CRUD + paging/sorting; nhưng vẫn phải hiểu abstraction thay vì gọi mọi method ở mọi nơi.

E. Bước 4 – Service: business rule + transaction

CategoryService.java

```
@Service
public class CategoryService {
    private final CategoryRepository categoryRepository;

    public CategoryService(CategoryRepository categoryRepository) {
        this.categoryRepository = categoryRepository;
    }

    @Transactional
    public Category createCategory(String name, String description) {
        String normalized = name == null ? "" : name.trim();
        if (normalized.isBlank()) {
            throw new IllegalArgumentException("Category name is required");
        }
        if (categoryRepository.existsByNameIgnoreCase(normalized)) {
            throw new IllegalArgumentException("Category already exists");
        }
        return categoryRepository.save(new Category(normalized, description));
    }

    @Transactional(readOnly = true)
    public List<Category> getActiveCategories() {
        return categoryRepository.findByActiveTrueOrderByNameAsc();
    }
}
```

Logic	Layer phù hợp	Vì sao
HTTP status / path param	Controller	Concern của transport/web
Check category name duplicate	Service	Business rule/use case
findByNameIgnoreCase	Repository	Data access query
Foreign key & unique constraint	Database + entity mapping	Data integrity cuối cùng
Mapping Entity -> DTO	Service/mapper boundary	Không expose persistence model trực tiếp ra API

F. Bước 5 – Many-to-One / One-to-Many

News.java – owning side

```
@Entity
@Table(name = "news")
public class News {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    public void setCategory(Category category) {
        this.category = category;
    }
}
```

Category.java – inverse side tối thiểu

```
@OneToMany(mappedBy = "category")
private List<News> newsItems = new ArrayList<>();
```

Part A scope

Ở Slot 17, mục tiêu là hiểu mapping semantics. Không bắt buộc biến mọi relation thành bidirectional. Nếu ứng dụng chỉ cần điều hướng News -> Category, mapping Many-to-One unidirectional thường đơn giản và đủ.

Break-It 2 – mappedBy

Mục tiêu: Phân biệt Java field với database column.

- Đổi mappedBy="category" thành mappedBy="category_id".
- Run app và đọc lỗi.
- Giải thích vì sao mappedBy phải trỏ tới field/property Java ở owning side, không phải tên FK column.

Bảng chứng cần lưu: Root cause + bản sửa + 2 câu giải thích.

G. Bước 6 – Many-to-Many và entity trung gian**News.java – direct Many-to-Many**

```
@ManyToMany
@JoinTable(
    name = "news_tags",
    joinColumns = @JoinColumn(name = "news_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

Tình huống	Thiết kế nên chọn
Chỉ cần liên kết News <-> Tag, không có metadata	@ManyToMany trực tiếp có thể đủ
Cần addedAt / addedBy / relevanceScore	Tạo NewsTag entity với 2 Many-to-One
Quan hệ có lifecycle riêng hoặc business rules	Entity trung gian rõ ràng hơn
Cần audit/history	Entity trung gian thường phù hợp hơn

H. Bước 7 – Seed data và verify SQL**Seed data demo**

```
@Bean
CommandLineRunner seed(CategoryRepository categoryRepository) {
    return args -> {
        if (categoryRepository.count() == 0) {
            categoryRepository.save(new Category("Technology", "Tech news"));
            categoryRepository.save(new Category("Education", "Education news"));
        }
    };
}
```

- Quan sát INSERT statements trong console.
- Restart ứng dụng với H2 memory + create-drop để thấy data lifecycle.

- Chạy derived query và đọc SQL WHERE/ORDER BY được sinh ra.
- Không đánh giá mapping chỉ bằng việc “app không lỗi”; phải kiểm tra schema, constraints và query behavior.

I. Bước 8 – Transaction trace workshop

Step	Điều gì xảy ra	Điểm cần quan sát
1	Controller/use case gọi Service	Boundary bắt đầu ở method @Transactional
2	Repository load Category	Entity trở thành managed trong persistence context
3	Service thay đổi name	Object thay đổi trong memory
4	Commit/flush	Hibernate sinh UPDATE nếu dirty
5	Transaction kết thúc	Entity có thể trở thành detached ngoài context

Trace 1 – Dirty Checking

Mục tiêu: Chứng minh update không chỉ là “save mọi lúc”.

- Trong @Transactional method: find entity -> setName(...).
- Tạm thời không gọi save() lần hai.
- Quan sát UPDATE khi commit.
- Giải thích điều kiện: entity phải managed và transaction/persistence context phù hợp.

Bảng chứng cần lưu: Console SQL + giải thích 4 bước.

J. Bước 9 – JSON recursion awareness và DTO boundary

Khi entity có quan hệ bidirectional, serialize trực tiếp entity graph ra JSON có thể tạo recursion, payload quá lớn hoặc vô tình expose persistence internals. Slot 17 chưa đi sâu DTO mapping, nhưng sinh viên phải hiểu rằng Entity ≠ API Response DTO.

Không nên	Nên
Controller trả thẳng object graph lớn chỉ vì “Jackson serialize được”	Thiết kế response DTO theo API contract
Dùng annotation JSON để chữa mọi mapping problem	Tách persistence model và API model
Fetch EAGER để “hết LazyInitialization”	Xác định use case/query boundary đúng

Design Review – DTO Boundary

Mục tiêu: Phân biệt persistence model và API contract.

- Chọn News entity có Category và Tags.
- Thiết kế NewsSummaryResponse chỉ gồm id, title, categoryName, tagNames.
- Giải thích vì sao response không cần expose toàn bộ Category/Tag entities.

Bảng chứng cần lưu: DTO sketch + mapping pseudocode.

K. Bước 10 – Testing JPA slice ở mức nhận biết

Repository test preview

```
@DataJpaTest
class CategoryRepositoryTest {
    @Autowired CategoryRepository categoryRepository;
```

```

@Test
void shouldFindCategoryIgnoringCase() {
    categoryRepository.save(new Category("Technology", "Tech"));

    Optional<Category> result =
        categoryRepository.findByNameIgnoreCase("technology");

    assertThat(result).isPresent();
    assertThat(result.get().getName()).isEqualTo("Technology");
}
}

```

Liên kết Slot 16

@DataJpaTest tập trung vào JPA components/repositories. Đây là lúc annotation đó có ngữ cảnh đúng; với repository ArrayList của Lab 03 core thì plain JUnit phù hợp hơn.

L. PA02 – PROJECT ASSISTANT 02: DATABASE / JPA READINESS CHECKPOINT

Theo yêu cầu Slot 17, phần cuối buổi dành cho PA02 nhằm chuyển kiến thức JPA vừa học vào Group Project. Nếu lịch lớp đã thực hiện PA02 ở Slot 16, phần này dùng như PA02 continuation/technical remediation trước Phase database integration.

Nhóm phải mang theo	Giảng viên kiểm tra	Bảng chứng
ERD 5+ entities	PK/FK, 1-N, N-N có hợp lý?	ERD versioned trong repo
API contract	Resource có khớp domain entities/DTO?	Endpoint matrix
Entity mapping draft	@Entity/@Id/relationships/owning side	Code compile + schema generated
Repository plan	JpaRepository + query methods cần thiết	Repository interfaces
Service plan	Business rules + transaction candidates	Service/use-case list
Git evidence	Ai làm module nào? commit/branch?	Git log + PR/issue

PA02 Quality Gate

Gate	PASS khi	BLOCK khi
G1 Domain	Entity/relationship phản ánh business rule	5 table được tạo “cho đủ số” nhưng không có nghiệp vụ
G2 Ownership	Giải thích được FK nằm đầu và owning side	Chỉ nhớ annotation nhưng không giải thích mappedBy
G3 Repository	Query methods gắn use case	Repository chứa business logic
G4 Service	Có use-case/service boundary rõ	Controller gọi repository trực tiếp ở mọi nơi
G5 Evidence	Code + ERD + Git khớp nhau	ERD một kiểu, code một kiểu, commit dồn cuối

V. KINH NGHIỆM THỰC TIỄN, LỖI THƯỜNG GẶP VÀ DEBUGGING

Triệu chứng	Nguyên nhân thường gặp	Cách kiểm tra
Not a managed type	Entity ngoài component scan / thiếu @Entity	Package structure, annotation
No identifier specified	Thiếu @Id	Entity mapping

Triệu chứng	Nguyên nhân thường gặp	Cách kiểm tra
mappedBy unknown property	mappedBy dùng tên cột hoặc sai field	Tên Java field ở owning side
Constraint violation	nullable/unique/FK không thỏa	SQL error + DB constraints
Infinite JSON recursion	Expose bidirectional entity graph	DTO boundary / serialization
LazyInitialization-like issue	Access LAZY ngoài persistence context	Transaction/use-case/query design
N+1 queries	Loop truy cập association gây nhiều SELECT	Bật SQL log, đếm query
Duplicate relation rows	equals/hashCode/Set/list/mapping sai	Join table + object collection

5.1. Debugging decision tree

Decision tree

App không start?
 -> đọc ROOT CAUSE
 -> entity/id mapping?
 -> datasource/driver?
 -> relation/mappedBy?

App start nhưng query sai?
 -> bật SQL log
 -> schema/constraint đúng?
 -> method name query đúng property?
 -> transaction/fetch boundary đúng?

REST response lỗi/recursion?
 -> xem entity graph có bidirectional relation?
 -> dùng DTO boundary thay vì chứa bằng EAGER khắp nơi

5.2. Best practices ở mức Part A

- Thiết kế ERD/business rule trước annotation.
- Ưu tiên constructor injection cho Service.
- Repository tập trung data access; Service tập trung use case/business logic.
- Không mặc định CascadeType.ALL hoặc FetchType.EAGER cho mọi association.
- Không expose entity trực tiếp làm API contract dài hạn.
- Viết helper methods để đồng bộ hai phía của bidirectional association nếu dùng.
- Đọc SQL được sinh ra; ORM không thay thế kiến thức SQL.
- Mỗi mapping change phải có bằng chứng: schema/query/test/Git commit.

VI. BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

Bài 1 – Entity cơ bản

Mục tiêu: Tạo Department entity có id, name unique, active.

- Tạo Department entity có id, name unique, active.

Bằng chứng cần lưu: Entity compile; H2 tạo table đúng; chụp schema.

Bài 2 – Repository

Mục tiêu: Tạo DepartmentRepository extends JpaRepository và findByNameIgnoreCase.

- Tạo DepartmentRepository extends JpaRepository và findByNameIgnoreCase.

Bằng chứng cần lưu: Test query với 2 case khác nhau.

Bài 3 – Service rule

Mục tiêu: Không cho tạo Department trùng tên.

- Không cho tạo Department trùng tên.

Bảng chứng cần lưu: Business rule nằm ở Service; test duplicate.

Bài 4 – Many-to-One

Mục tiêu: Employee thuộc Department.

- Employee thuộc Department.

Bảng chứng cần lưu: FK department_id nằm đúng table Employee.

Bài 5 – Bidirectional relation

Mục tiêu: Department có employees với mappedBy.

- Department có employees với mappedBy.

Bảng chứng cần lưu: Giải thích owning side bằng lời, không chỉ code.

Bài 6 – Many-to-Many

Mục tiêu: Student–Course hoặc News–Tag.

- Student–Course hoặc News–Tag.

Bảng chứng cần lưu: Join table đúng; test add relation.

Bài 7 – Association entity

Mục tiêu: Thay Many-to-Many Student–Course bằng Enrollment có enrolledAt/grade.

- Thay Many-to-Many Student–Course bằng Enrollment có enrolledAt/grade.

Bảng chứng cần lưu: ERD + 3 entity classes/pseudocode.

Bài 8 – Fetch experiment

Mục tiêu: So sánh query khi truy cập association LAZY/EAGER trong demo nhỏ.

- So sánh query khi truy cập association LAZY/EAGER trong demo nhỏ.

Bảng chứng cần lưu: SQL log + nhận xét, không kết luận “EAGER luôn tốt”.

Bài 9 – DTO boundary

Mục tiêu: Thiết kế response DTO cho entity có relationships.

- Thiết kế response DTO cho entity có relationships.

Bảng chứng cần lưu: Không serialize object graph toàn bộ.

Bài 10 – PA02 transfer

Mục tiêu: Áp dụng cho 2 entity thật của Group Project.

- Áp dụng cho 2 entity thật của Group Project.

Bằng chứng cần lưu: ERD + code + repository + service + Git evidence.

6.1. Bài tập tổng hợp – FUNews Persistence Core

Deliverable	Yêu cầu tối thiểu
Entities	Category, News, Tag; có PK/constraints hợp lý
Relationships	Category 1-N News; News N-N Tag
Repositories	JpaRepository cho 3 entities + ít nhất 3 derived query methods
Services	CategoryService + NewsService có business rule rõ
Verification	H2 schema + SQL log + repository test
Design note	Owning side, cascade/fetch decision, DTO boundary
Git	Issue -> branch -> commits -> PR/merge evidence

VII. FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa trong bài	Ví dụ
JPA / Jakarta Persistence	Specification/API cho persistence và ORM mapping	@Entity, EntityManager
ORM	Ánh xạ object <-> relational data	News object <-> news row
Hibernate	JPA provider/ORM implementation	Sinh SQL, persistence context
Spring Data JPA	Repository abstraction trên JPA	JpaRepository
Entity	Domain object có persistent identity	Category
Persistence Context	Context quản lý entity instances	Managed entity, dirty checking
Transient	Entity object chưa được quản lý	new Category(...)
Managed	Entity đang được persistence context theo dõi	Loaded trong transaction
Detached	Entity không còn được context quản lý	Sau context close
Primary Key	Định danh row/entity	@Id
Foreign Key	Ràng buộc tham chiếu giữa tables	news.category_id
@Entity	Đánh dấu persistent entity	Category class
@Table	Tùy biến table mapping	@Table(name="categories")
@Id	Đánh dấu identifier	Long id
@GeneratedValue	Sinh identifier	IDENTITY
@Column	Mapping/basic constraint cho column	nullable=false
JpaRepository	JPA repository interface với CRUD/paging/sorting	CategoryRepository
Derived Query	Query suy ra từ method name	findByNameIgnoreCase

Thuật ngữ	Định nghĩa trong bài	Ví dụ
@Service	Spring stereotype cho service layer	CategoryService
@Transactional	Định nghĩa transaction boundary	create/update use case
Dirty Checking	Theo dõi thay đổi managed entity	setName -> UPDATE at flush
@ManyToOne	N entity con tham chiếu 1 parent	News -> Category
@OneToMany	1 parent có collection child	Category -> News
@ManyToMany	N <-> N association	News <-> Tag
@JoinColumn	Column FK cho association	category_id
@JoinTable	Join table cho association	news_tags
mappedBy	Field ở owning side mà inverse side tham chiếu	mappedBy="category"
Owning Side	Side điều khiển DB association mapping	News.category
Inverse Side	Side dùng mappedBy trong bidirectional relation	Category.newsItems
Cascade	Propagate persistence operations	PERSIST/REMOVE...
orphanRemoval	Xóa child khi bị loại khỏi owned collection	Parent-owned child lifecycle
FetchType.LAZY	Trì hoãn lấy association đến khi cần (hint tùy context)	OneToMany default classic behavior
FetchType.EAGER	Yêu cầu association được fetch sớm	Cẩn trọng over-fetching
N+1 Query	1 query parent + N query associations	Performance smell
DTO	Object contract cho API/boundary	NewsSummaryResponse
@DataJpaTest	JPA test slice	Repository test

VIII. RESEARCH BASE VÀ AI-BASED LEARNING

8.1. Nguyên tắc dùng AI

AI may generate; student must verify

Mọi câu trả lời/code do AI tạo phải được kiểm chứng bằng official docs, compile/run, SQL log/test và khả năng giải thích. Không dùng “AI nói vậy” làm bằng chứng kỹ thuật.

A. 10 prompt chuyên sâu kỹ thuật

- Phân tích lifecycle của entity JPA: transient, managed, detached, removed và ảnh hưởng đến save/update.
- Giải thích persistence context và dirty checking bằng một trace SQL cụ thể.
- So sánh JpaRepository, CrudRepository và Repository về API surface và use case.
- Giải thích owning side và inverse side trong bidirectional OneToMany/ManyToOne.
- Phân tích khi nào orphanRemoval=true phù hợp và khi nào nguy hiểm.
- Phân tích cascade PERSIST/MERGE/REMOVE và vì sao CascadeType.ALL không nên dùng theo thói quen.
- Giải thích N+1 query problem và cách nhận diện bằng SQL log.
- So sánh ManyToMany trực tiếp với association entity có thuộc tính riêng.
- Giải thích why Entity != DTO và rủi ro khi serialize entity graph trực tiếp.
- Phân tích transaction boundary ở Service khi một use case gọi nhiều repository.

B. 10 prompt thực tiễn production

14. Trong production, chiến lược schema migration nên thay ddl-auto=create/update như thế nào?
15. Làm sao thiết kế auditing createdAt/updatedAt/createdBy cho JPA entities?
16. Khi nào nên dùng optimistic locking với @Version?
17. Khi nào query method name trở nên quá phức tạp và nên chuyển @Query/Specification?
18. Làm sao tránh LazyInitializationException mà không chuyển tất cả sang EAGER?
19. Làm sao thiết kế pagination cho entity có nhiều relationships mà vẫn tránh duplicate rows?
20. Trong domain lớn, aggregate boundary ảnh hưởng cascade và repository design thế nào?
21. Làm sao test JPA mapping với database gần production hơn H2?
22. Làm sao quản lý transaction khi tích hợp external API và database update?
23. Làm sao review JPA code để phát hiện performance smell trước production?

C. 10 prompt cập nhật công nghệ/xu hướng

24. So sánh Jakarta Persistence 3.x/4.x về fetch configuration và các thay đổi đáng chú ý.
25. Spring Data JPA 3.5 và 4.x khác nhau ở những điểm nào quan trọng với repository API?
26. Hibernate ORM 6.x và 7.x có những thay đổi gì ảnh hưởng mapping/query?
27. So sánh JPA/Hibernate với jOOQ cho ứng dụng cần SQL control cao.
28. So sánh JPA với Spring Data JDBC trong domain đơn giản.
29. Khi nào dùng projections/entity graphs để kiểm soát fetch?
30. Keyset pagination trong Spring Data hiện đại khác offset pagination thế nào?
31. AOT/native image ảnh hưởng Spring Data JPA/repositories thế nào?
32. AI coding assistant thường tạo sai JPA mappings ở những pattern nào?
33. Thực hành tốt hiện nay cho database integration test với Testcontainers là gì?

8.2. AI Verification Worksheet

Prompt / claim của AI	Nguồn official kiểm chứng	Experiment/Test	Kết luận của tôi
Ví dụ: "ManyToOne luôn EAGER"	Jakarta Persistence docs / course baseline	Bật SQL log, chạy use case	Nêu rõ default/course version và decision thực tế

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

9.1. Knowledge self-check

- JPA, Hibernate và Spring Data JPA khác nhau thế nào?
- Entity cần identity vì sao?
- JpaRepository được implement ở đâu? Sinh viên có cần viết implementation CRUD không?
- Business rule nên đặt ở Service hay Repository? Vì sao?
- Foreign key nằm ở đâu trong ManyToOne/OneToMany?
- mappedBy trỏ tới gì?
- Owning side là gì?
- Cascade và orphanRemoval khác nhau thế nào?
- Vì sao không nên đặt EAGER cho mọi relationship?
- ManyToMany khi nào nên thay bằng association entity?
- Entity state và dirty checking là gì?
- Vì sao Entity không nên đồng nhất với API DTO?

9.2. Skill checklist

Năng lực	Đạt	Chưa đạt	Bảng chứng
----------	-----	----------	------------

Năng lực	Đạt	Chưa đạt	Bảng chứng
Tạo entity có id/constraints đúng			
Tạo JpaRepository và derived query			
Tạo Service với business rule			
Map ManyToOne / OneToMany			
Giải thích owning side/mappedBy			
Map ManyToMany hoặc association entity			
Quan sát schema/SQL sinh ra			
Viết @DataJpaTest cơ bản			
Chuyển ERD Group Project thành mapping draft			
Giải thích code không nhìn AI/chat history			

9.3. Ba mức tự đánh giá

Mức	Biểu hiện
Mức 1 – Chưa đạt	Chỉ nhớ annotation; không giải thích được FK/owning side; project chưa generate schema đúng.
Mức 2 – Đạt cơ bản	Tạo được entity/repository/service, mapping 1-N và query cơ bản; đọc được SQL log.
Mức 3 – Tốt	Giải thích được trade-off cascade/fetch/association entity, transaction boundary và áp dụng vào ERD project có bằng chứng test/Git.

PHỤ LỤC A – QUICK REFERENCE: JPA ANNOTATIONS

Annotation/API	Dùng khi	Ghi nhớ
@Entity	Class được persist	Cần identifier
@Id	Primary key	Một entity phải có identity
@GeneratedValue	Sinh id	Strategy phụ thuộc DB/design
@Column	Constraint/mapping basic field	nullable/unique/length không thay business validation
@ManyToOne	Child -> parent	Thường là owning side có FK
@OneToMany	Parent -> children	mappedBy khi bidirectional
@ManyToMany	N <-> N	Join table; cân nhắc association entity
@JoinColumn	FK column	Tên column DB
@JoinTable	Join table	joinColumns/inverseJoinColumns
JpaRepository<T,ID>	Repository JPA	CRUD + paging/sorting
@Service	Business/use-case layer	Constructor injection
@Transactional	Transaction boundary	Service-level use case thường là vị trí tự nhiên

Annotation/API	Dùng khi	Chi nhớ
@DataJpaTest	JPA slice test	Repository/mapping test

PHỤ LỤC B – RELATIONSHIP DESIGN WORKSHEET

Relationship	Cardinality	FK/Join table	Owning side	Inverse side	Cascade?	Fetch?	Lý do
Example: News–Category	N:1	news.category_id	News.category	Category.newsItems	No/limited	LAZY	Category shared

PHỤ LỤC C – HUMAN VERIFICATION GATE

34. Vẽ lại schema được Hibernate tạo từ 2 entity bạn chọn.
35. Chỉ vào field nào tạo foreign key và giải thích.
36. Nếu bỏ mappedBy thì schema có thể thay đổi thế nào?
37. Tại sao repository interface không có implementation class do bạn viết?
38. Nếu đổi findByNameIgnoreCase thành findByTitleIgnoreCase trong CategoryRepository thì chuyện gì xảy ra?
39. Tại sao business rule duplicate name không chỉ dựa vào UI?
40. Một entity managed thay đổi field trong transaction có cần save lần hai không? Giải thích điều kiện.
41. Khi nào CascadeType.REMOVE nguy hiểm?
42. Khi nào orphanRemoval=true hợp lý?
43. Nếu News–Tag cần addedAt, bạn đổi model thế nào?
44. N+1 query là gì? Bạn sẽ phát hiện bằng cách nào?
45. Vì sao EAGER không phải thuốc chữa LazyInitialization?
46. Entity và DTO khác nhau ở boundary nào?
47. Nếu database có unique constraint nhưng Service không check trước, hệ thống vẫn có thể bảo vệ integrity không? Điều gì xảy ra?
48. Sửa trực tiếp một relationship trong project và dự đoán DDL/SQL thay đổi trước khi run.
49. Giảng viên chọn ngẫu nhiên một annotation trong code: giải thích semantics, không chỉ đọc tên.
50. Member A giải thích mapping do Member B viết.
51. Từ một ERD mới, hãy chọn owning side trong 2 phút.
52. Thay ManyToMany bằng association entity ngay trên giấy và liệt kê file cần sửa.
53. Giải thích transaction boundary cho use case tạo News + gắn Tags.

PHỤ LỤC D – PA02 PRE-CHECKPOINT REPORT TEMPLATE

Slide/Phần	Nội dung bắt buộc
1. Team & scope	Tên nhóm, đề tài, thành viên, link Git
2. ERD	5+ entities, PK/FK, 1-N/N-N, constraints
3. Mapping draft	2–3 entity code, owning side/mappedBy
4. Repository + Service	Repository methods và business use cases
5. Evidence	Schema/SQL/test + commit/PR
6. Blockers	JPA mapping issues, DB questions, next actions

PHỤ LỤC E – EVIDENCE PACKAGE SLOT 17

ID	Bằng chứng	Mục đích
E01	Pre-class 3–2–1 note	Preparation
E02	Entity design worksheet	Reasoning before code
E03	Generated schema screenshot	Mapping verification
E04	SQL log for CRUD/query	Runtime evidence
E05	Repository test result	Data-access verification
E06	Relationship ERD	Design traceability
E07	Break-It mappedBy evidence	Debugging skill
E08	Transaction trace	Persistence-context understanding
E09	DTO boundary sketch	Architecture awareness
E10	PA02 mapping draft	Project transfer
E11	Git issue/branch/commits	Process evidence
E12	AI Verification Log	AI-assisted integrity

PHỤ LỤC F – KẾ HOẠCH DẠY 135 PHÚT

Thời lượng	Hoạt động	Sản phẩm
0–10	Retrieval: JPA/ORM/DB fundamentals + misconception poll	3–2–1 + quick answers
10–30	Mini-lecture: Entity, repository, persistence context	Mental model notes
30–55	Guided coding: Category entity + repository + H2 verify	Schema + SQL log
55–80	Relationships: News–Category, News–Tag	ERD + mapping code
80–100	Service + transaction trace + dirty checking experiment	Trace worksheet
100–115	Break-It Lab: mappedBy / constraint / fetch observation	Debug evidence
115–130	PA02 project mapping review	ERD + mapping draft
130–135	Exit ticket + Human Gate sample	Individual response

PHỤ LỤC G – TECHNICAL UPDATE 2026 & REFERENCES

- Spring Data JPA Reference Documentation – repository abstraction, JPA repositories, query methods and persistence support: <https://docs.spring.io/spring-data/jpa/reference/>
- Spring Data JPA Getting Started – entity/repository examples and repository auto-implementation: <https://docs.spring.io/spring-data/jpa/reference/jpa/getting-started.html>
- Hibernate ORM User Guide – domain model and association mappings: https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html
- Jakarta Persistence API – Entity and relationship annotations: <https://jakarta.ee/specifications/persistence/>
- Spring Boot 3.x course track: JDK 21 is supported; use Spring Boot dependency management rather than pinning Spring Data/Hibernate versions manually unless the course explicitly requires it.

Course-version note

Năm 2026, Spring Boot 4 / Spring Data 4 đã tồn tại. Tuy nhiên SBA301 đang dùng Spring Boot 3.x + JDK 21, vì vậy code/lab của Slot 17 giữ Jakarta namespace và dependency management tương thích với Boot 3.x. Các “current ecosystem” thay đổi được nêu như awareness, không ép sinh viên migrate giữa môn.

PHỤ LỤC H – WORKED EXAMPLE 1: TỪ ERD ĐẾN JPA CODE (PRODUCT CATALOG)

Mục đích

Ví dụ này đi trọn một đường thiết kế: business rule -> ERD -> ownership -> entity -> repository -> service -> SQL verification. Sinh viên nên đọc theo thứ tự và đối chiếu với một module thật trong Group Project.

H.1. Business rules trước khi vẽ ERD

- Một Category có thể chứa nhiều Product.
- Một Product bắt buộc thuộc đúng một Category tại một thời điểm.
- Category name không được trùng theo business rule.
- Product SKU phải unique; price không âm.
- Không cho xóa Category nếu còn Product active, trừ khi use case định nghĩa migration rõ ràng.

Business rule	Database implication	JPA implication
Product bắt buộc có Category	product.category_id NOT NULL + FK	@ManyToOne(optional=false) + @JoinColumn(nullable=false)
Category name unique	UNIQUE constraint	@Column(unique=true) + Service check
Product SKU unique	UNIQUE constraint	@Column(unique=true)
Không xóa Category có Product active	Không chỉ là FK constraint	Business rule ở Service; không lạm dụng cascade remove

ERD logic

```

Category(id PK, name UQ, active)
  1
  |
  | category_id FK NOT NULL
  N
Product(id PK, sku UQ, name, price, active, category_id FK)

```

H.2. Mapping code và ownership**Product.java**

```

@Entity
@Table(name = "products")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 40)
    private String sku;

    @Column(nullable = false, length = 150)
    private String name;

```

```

@Column(nullable = false, precision = 12, scale = 2)
private BigDecimal price;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "category_id", nullable = false)
private Category category;
}

```

Category.java

```

@Entity
@Table(name = "categories")
public class Category {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @OneToMany(mappedBy = "category")
    private List<Product> products = new ArrayList<>();
}

```

Trace ownership

Cột category_id nằm ở PRODUCTS. Vì Product.category quyết định giá trị FK, Product là owning side. Category.products chỉ là hướng điều hướng ngược trong object graph và dùng mappedBy="category".

H.3. Repository design

Repository interfaces

```

public interface ProductRepository extends JpaRepository<Product, Long> {
    Optional<Product> findBySkuIgnoreCase(String sku);
    boolean existsBySkuIgnoreCase(String sku);
    Page<Product> findByCategoryIdAndActiveTrue(Long categoryId, Pageable pageable);
}

public interface CategoryRepository extends JpaRepository<Category, Long> {
    boolean existsByNameIgnoreCase(String name);
}

```

Query method	Use case	Review question
findBySkuIgnoreCase	Lookup product theo business key	Nếu SKU normalized uppercase thì còn cần IgnoreCase không?
existsBySkuIgnoreCase	Pre-check duplicate	Unique constraint DB vẫn cần vì race condition có thể xảy ra
findByCategoryIdAndActiveTrue	List/filter/paging	Index nào có thể hữu ích ở DB?

H.4. Service use case và data integrity

Create product use case

```

@Service
public class ProductService {
    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;
}

```

```

public ProductService(ProductRepository productRepository,
    CategoryRepository categoryRepository) {
    this.productRepository = productRepository;
    this.categoryRepository = categoryRepository;
}

@Transactional
public Product createProduct(String sku, String name,
    BigDecimal price, Long categoryId) {
    if (price == null || price.signum() < 0) {
        throw new IllegalArgumentException("Price must be >= 0");
    }
    if (productRepository.existsBySkuIgnoreCase(sku)) {
        throw new IllegalArgumentException("SKU already exists");
    }
    Category category = categoryRepository.findById(categoryId)
        .orElseThrow(() -> new IllegalArgumentException("Category not found"));

    Product product = new Product(sku.trim(), name.trim(), price, category);
    return productRepository.save(product);
}
}

```

- Validation annotation có thể bảo vệ input shape; business validation vẫn cần Service.
- Unique constraint ở DB là lớp bảo vệ integrity cuối cùng; pre-check chỉ giúp thông báo lỗi thân thiện hơn.
- Transaction boundary bao phủ cả việc load Category và persist Product trong một use case.

Worked Exercise H1

Mục tiêu: Chuyển một quan hệ 1-N trong Group Project thành code JPA.

- Viết 3 business rules.
- Vẽ FK và chọn owning side.
- Tạo 2 entity classes + repository interfaces.
- Viết 1 Service use case có transaction.
- Dự đoán SQL INSERT/SELECT trước khi run.

Bằng chứng: ERD, code, SQL log, commit.

PHỤ LỤC I – WORKED EXAMPLE 2: ASSOCIATION ENTITY THAY CHO MANY-TO-MANY

Tư duy domain

Nhiều hệ thống thực tế không có “Many-to-Many thuần”. Khi quan hệ có dữ liệu/behavior riêng, join table thực chất là một entity.

I.1. Ví dụ Student–Course

Từ join table đến domain entity

Student N ---- N Course

Nếu chỉ liên kết: student_courses(student_id, course_id)

Nếu cần:

- enrolled_at
- status
- final_grade

- enrolled_by

=> Enrollment là entity riêng.

Enrollment.java

```
@Entity
@Table(name = "enrollments",
      uniqueConstraints = @UniqueConstraint(columnNames = {"student_id", "course_id"}))
public class Enrollment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "student_id", nullable = false)
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "course_id", nullable = false)
    private Course course;

    @Column(nullable = false)
    private LocalDateTime enrolledAt;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private EnrollmentStatus status;

    private BigDecimal finalGrade;
}
```

Direct @ManyToMany	Association entity
Code ngắn, phù hợp relation rất đơn giản	Thêm được metadata/business rules
Join table bị ẩn sau mapping	Join table trở thành domain concept rõ
Khó gắn lifecycle/audit cho relation	Có repository/service/query riêng
Dễ bị lạm dụng vì tiện	Thường dễ mở rộng hơn trong hệ thống nghiệp vụ

Worked Exercise I1

Mục tiêu: Nhận diện một N-N trong project có nên trở thành entity trung gian.

- Chọn một relation N-N.
- Liệt kê ít nhất 3 thuộc tính có thể thuộc về relation.
- Nếu có thuộc tính đáng kể, vẽ lại thành 2 quan hệ N-1.
- Nêu business rules thuộc association entity.

Bảng chứng: Before/after ERD + rationale.

PHỤ LỤC J – ENTITY DESIGN DEEP DIVE

J.1. ID generation strategy – học semantics, không học thuộc

Strategy	Ý tưởng	Lưu ý dạy học
IDENTITY	DB identity/auto-increment thường sinh id khi INSERT	Đơn giản với SQL Server; ảnh hưởng batching/when id available tùy provider

Strategy	Ý tưởng	Lưu ý dạy học
SEQUENCE	Dùng database sequence	Phù hợp DB hỗ trợ sequence; có thể tối ưu allocation
AUTO	Provider chọn strategy	Tiện demo nhưng behavior phụ thuộc DB/provider
UUID	Identifier dạng UUID	Hữu ích distributed scenarios nhưng có trade-off index/storage

Course choice

Với demo H2/SQL Server cơ bản, IDENTITY dễ hiểu. Tuy nhiên sinh viên phải biết đây là một design choice, không phải annotation cố định cho mọi hệ thống.

J.2. Field types và mapping decisions

Java type	Use case	Gợi ý
BigDecimal	Money/precise decimal	Tránh double cho tiền; khai báo precision/scale hợp lý
LocalDate	Ngày không có timezone	Birthday, due date
LocalDateTime	Timestamp local	createdAt demo; production cần hiểu timezone policy
Instant	Absolute timestamp	Audit/event time xuyên timezone
Enum	Finite domain values	EnumType.STRING thường dễ đọc và ổn định hơn ORDINAL
boolean/Boolean	Required vs nullable flag	Chọn theo business optionality

J.3. Nullability có nhiều lớp

Layer	Ví dụ	Vai trò
UI/Form	Required field	UX feedback sớm
Bean Validation	@NotBlank / @Positive	Validate request/application input
JPA metadata	@Column(nullable=false)	Schema generation hint/metadata
Database	NOT NULL / CHECK / FK / UNIQUE	Data integrity cuối cùng
Service	Cross-field/business rule	Ví dụ endDate >= startDate

Không thay thế lẫn nhau

Một lớp validation không làm các lớp khác “thừa”. Hệ thống tốt dùng defense in depth: UX -> validation -> business rule -> database constraint.

J.4. equals/hashCode awareness

- Entity equality khó hơn POJO thông thường vì id có thể null trước khi persist.
- Không nên generate equals/hashCode chứa toàn bộ relationships bằng Lombok một cách mù quáng; có thể gây recursion hoặc collection behavior khó đoán.
- Trong Part A, ưu tiên hiểu vấn đề và tránh dùng entity làm Set key phức tạp trước khi có strategy rõ.

Break-It J1 – Enum ordinal

Mục tiêu: Quan sát vì sao ORDINAL có thể gây rủi ro khi enum thay đổi thứ tự.

- Lưu enum bằng ORDINAL trong demo.
- Đổi thứ tự enum constants.
- Giải thích data cũ có thể bị diễn giải sai thế nào.
- Chuyển sang STRING và so sánh.

Bằng chứng: Code + schema/data note.

PHỤ LỤC K – REPOSITORY QUERY DESIGN GUIDE

K.1. Từ use case đến query method

Use case	Repository signature gợi ý	Khi nào cần nâng cấp
Find by unique email	Optional<User> findByEmailIgnoreCase(String email)	Nếu search fuzzy -> custom query/specification
Check duplicate SKU	boolean existsBySkulgnoreCase(String sku)	DB unique constraint vẫn cần
List active products by category	Page<Product> findByCategoryIdAndActiveTrue(Long id, Pageable p)	Nếu filter động nhiều field -> Specification/query object
Top recent news	List<News> findTop10ByActiveTrueOrderByPublishedAtDesc()	Nếu nhiều criteria -> @Query/projection
Count appointments by status	long countByStatus(Status status)	Analytics phức tạp -> aggregate query/DTO projection

K.2. Derived query method naming mistakes

- Tên property phải khớp Java property path, không phải tên column DB.
- Method name quá dài là tín hiệu use case/query đang phức tạp; đừng biến method name thành “ngôn ngữ lập trình thứ hai”.
- Optional<T> phù hợp với zero-or-one; List<T>/Page<T> phù hợp multi-result.
- Repository không nên trả DTO tùy tiện nếu chưa xác định projection/boundary rõ.

Naming smell

```
// Reasonable
Optional<User> findByEmailIgnoreCase(String email);
Page<Product> findByCategoryIdAndActiveTrue(Long categoryId, Pageable pageable);

// Smell: quá nhiều criteria trong tên
findByNameContainingIgnoreCaseAndCategoryIdAndPriceBetweenAndActiveTrueOrderByCreatedAtDesc(...)

// Khi filter trở nên động/phức tạp: cân nhắc @Query, Specification,
// Querydsl hoặc query service phù hợp ở slot/nội dung nâng cao.
```

K.3. Query verification worksheet

Use case	Method signature	Expected SQL idea	Index/constraint cần nghĩ	Test

Use case	Method signature	Expected SQL idea	Index/constraint cần nghĩ	Test

PHỤ LỤC L – SERVICE & TRANSACTION DESIGN WORKBOOK

L.1. Service không phải “repository wrapper”

Service method kiểu yếu	Service use case tốt hơn
repository.save(dtoToEntity(dto))	createProduct(...) kiểm tra category, duplicate SKU, price, audit...
repository.deleteById(id)	deleteCategory(id) kiểm tra active products / policy trước khi delete
repository.findAll()	listProducts(filter, pageable) thể hiện use case và query strategy
Controller gọi 3 repository liên tiếp	Service @Transactional điều phối toàn use case

L.2. Transaction scenario – Transfer-like use case

Business operation spanning repositories

```
@Transactional
public void moveProduct(Long productId, Long targetCategoryId) {
    Product product = productRepository.findById(productId)
        .orElseThrow(...);
    Category target = categoryRepository.findById(targetCategoryId)
        .orElseThrow(...);

    if (!target.isActive()) {
        throw new BusinessException("Target category is inactive");
    }

    product.setCategory(target);
    // dirty checking persists FK change at flush/commit
}
```

- Nếu exception runtime xảy ra trước commit, transaction thường rollback theo Spring defaults.
- Transaction boundary ở Service giúp use case atomic hơn so với nhiều repository calls rải trong Controller.
- readOnly=true là optimization/intent marker cho read operations; không phải security control.

Trace L1 – Transaction rollback

Mục tiêu: Quan sát atomicity.

- Trong transaction, tạo entity A rồi cố tình throw RuntimeException trước entity B.
- Kiểm tra DB sau rollback.
- Lập lại khi catch exception sai chỗ hoặc tách transaction để thấy khác biệt.

Bằng chứng: SQL log + DB state + explanation.

L.3. Transaction boundary decision table

Use case	Nên đặt @Transactional ở đâu?	Lý do
Một repository CRUD đơn giản	Repository đã có transaction defaults; Service vẫn có thể làm facade	Giữ architecture nhất quán
Use case gọi nhiều repositories	Service method	Một business transaction

Use case	Nên đặt @Transactional ở đâu?	Lý do
Read aggregate nhiều relation	Service read method / query-specific approach	Giữ persistence context trong use case
Controller method	Thường không phải lựa chọn đầu tiên	Trộn web concern với transaction concern

PHỤ LỤC M – RELATIONSHIP MAPPING DECISION MATRIX & ANTI-PATTERNS

Câu hỏi	Nếu YES	Nếu NO / Cảnh báo
Child có FK tới parent?	@ManyToOne thường là mapping tự nhiên	Đừng tạo @OneToMany join table chỉ vì muốn collection
Cần navigation parent -> children?	Thêm @OneToMany(mappedBy=...) nếu có use case	Không bắt buộc bidirectional
Relation N-N có metadata?	Tạo association entity	@ManyToMany direct có thể đủ nếu relation đơn giản
Child lifecycle hoàn toàn thuộc parent?	Cần nhắc cascade/orphanRemoval	Nếu child shared, REMOVE có thể nguy hiểm
Cần association trong mọi use case?	Có thể query/fetch chủ đích	Không biến tất cả thành EAGER
API cần toàn object graph?	Thường vẫn nên DTO/projection	Entity graph lớn dễ recursion/overfetch

M.1. Anti-pattern gallery

Anti-pattern	Biểu hiện	Rủi ro	Hướng đúng
Everything EAGER	Đặt mọi relation EAGER để tránh lazy issue	Over-fetching, query explosion, khó kiểm soát payload	Giữ mapping hợp lý; fetch theo use case/query.
Cascade ALL everywhere	CascadeType.ALL trên mọi relation	Xóa/merge lan ngoài ý muốn	Chọn cascade theo ownership/lifecycle.
Bidirectional by default	Relation nào cũng hai chiều	Object graph phức tạp, helper/serialization issues	Chỉ thêm chiều ngược khi có navigation use case.
Entity = API DTO	Controller trả entity trực tiếp	Leak schema, recursion, coupling	DTO/projection boundary.
Repository business logic	Repository quyết định discount/authorization	Data access bị trộn domain rule	Service/use-case layer.
ddl-auto update as migration	Dùng update ở mọi môi trường	Schema drift, khó review	Migration tooling ở production workflow.

PHỤ LỤC N – EXPERIMENT WORKBOOK (8 THÍ NGHIỆM BẮT BUỘC/GỢI Ý)

Experiment	Thao tác	Evidence
E1 – Schema Generation	Tạo Category/Product mapping và kiểm tra PK/FK/unique	DDL log + H2 schema
E2 – Derived Query	findByNamedIgnoreCase và findByActiveTrueOrderByNameAsc	SQL + test result
E3 – Dirty Checking	Load managed entity, sửa field không gọi save lần 2	UPDATE at commit
E4 – mappedBy Failure	Dùng mappedBy sai field rồi sửa	Startup root cause
E5 – Cascade Behavior	So sánh PERSIST có/không cascade trong parent-child demo	Insert behavior
E6 – Orphan Removal	Remove child khỏi collection với orphanRemoval true/false	DELETE behavior

Experiment	Thao tác	Evidence
E7 – Fetch Observation	Truy cập relation trong/ngoài use case; đếm SELECT	SQL trace
E8 – Rollback	Throw exception giữa multi-step transaction	DB state before/after

N.1. Experiment report template

Hypothesis	Code change	Observed SQL/behavior	Conclusion	Project decision

PHỤ LỤC O – PA02 PROJECT MAPPING PACK

Mục tiêu PA02

Không yêu cầu nhóm “code thật nhiều” trong buổi PA. Mục tiêu là chứng minh design nhất quán: business rule <-> ERD <-> entity relationships <-> repository queries <-> service use cases <-> API contract <-> Git ownership.

O.1. Module Mapping Sheet

Module	Business use case	Entities	Relationship	Repository methods	Service methods	Owner
Example Product	Create/list/filter Product	Product, Category	N:1	findByCategoryId...	createProduct/listProducts	Member 2

O.2. Relationship Review Sheet

Relation	Business sentence	FK location	Owning side	Need inverse side?	Cascade decision	Fetch decision	Reviewer

O.3. PA02 live questions for each member

54. Chỉ ra entity/module bạn chịu trách nhiệm và business rule chính.
55. Vẽ FK cho một relation trong module của bạn.
56. Giải thích owning side và mappedBy nếu có.
57. Chọn một repository method và giải thích use case/SQL idea.

58. Chọn một Service method và giải thích transaction boundary.
59. Nêu một lỗi mapping bạn từng gặp và bằng chứng debug.
60. Nếu thêm một field/relation mới, file nào phải thay đổi?
61. Member khác trong nhóm giải thích một relation do bạn viết.

O.4. PA02 decision log

Decision	Alternatives considered	Evidence	Reason	Owner	Status

PHỤ LỤC P – INSTRUCTOR / PEER TEACHING QUESTION BANK

Foundation

- JPA có phải Hibernate không?
- Repository implementation đến từ đâu?
- Entity khác DTO thế nào?
- Persistence context là gì?

Relationships

- FK của ManyToOne nằm ở đâu?
- mappedBy là tên field hay column?
- Tại sao bidirectional cần helper methods?
- Khi nào ManyToMany nên thành entity trung gian?

Performance awareness

- N+1 là gì?
- Vì sao EAGER có thể làm chậm?
- Làm sao kiểm tra số SQL query?
- Page query với relation lớn có rủi ro gì?

Transactions

- @Transactional nên ở Controller hay Service?
- Dirty checking cần điều kiện gì?
- Rollback xảy ra khi nào?
- readOnly=true có ý nghĩa gì?

Project transfer

- ERD hiện tại của nhóm có relation nào đáng nghi?
- Entity nào có lifecycle phụ thuộc parent?
- Relation nào đang dùng CascadeType.ALL và lý do?
- API response nào đang expose entity graph?